DOCUMENTO 01 · DATOS

Modelo Entidad-Relación y normalización 3FN

Nueve tablas, diez relaciones y un diccionario completo. El modelo separa la identidad del usuario de sus perfiles de rol, coloca al viaje como eje de la operación y aísla pagos, calificaciones y patrocinios para eliminar redundancias.

3FN verificada | 9 tablas | SQL incluido

01 Diagrama entidad-relación

Vista lógica completa. La línea punteada representa la clave foránea opcional con la que el administrador autoriza una liquidación.

FIGURA 1 · MODELO E-R LÓGICO

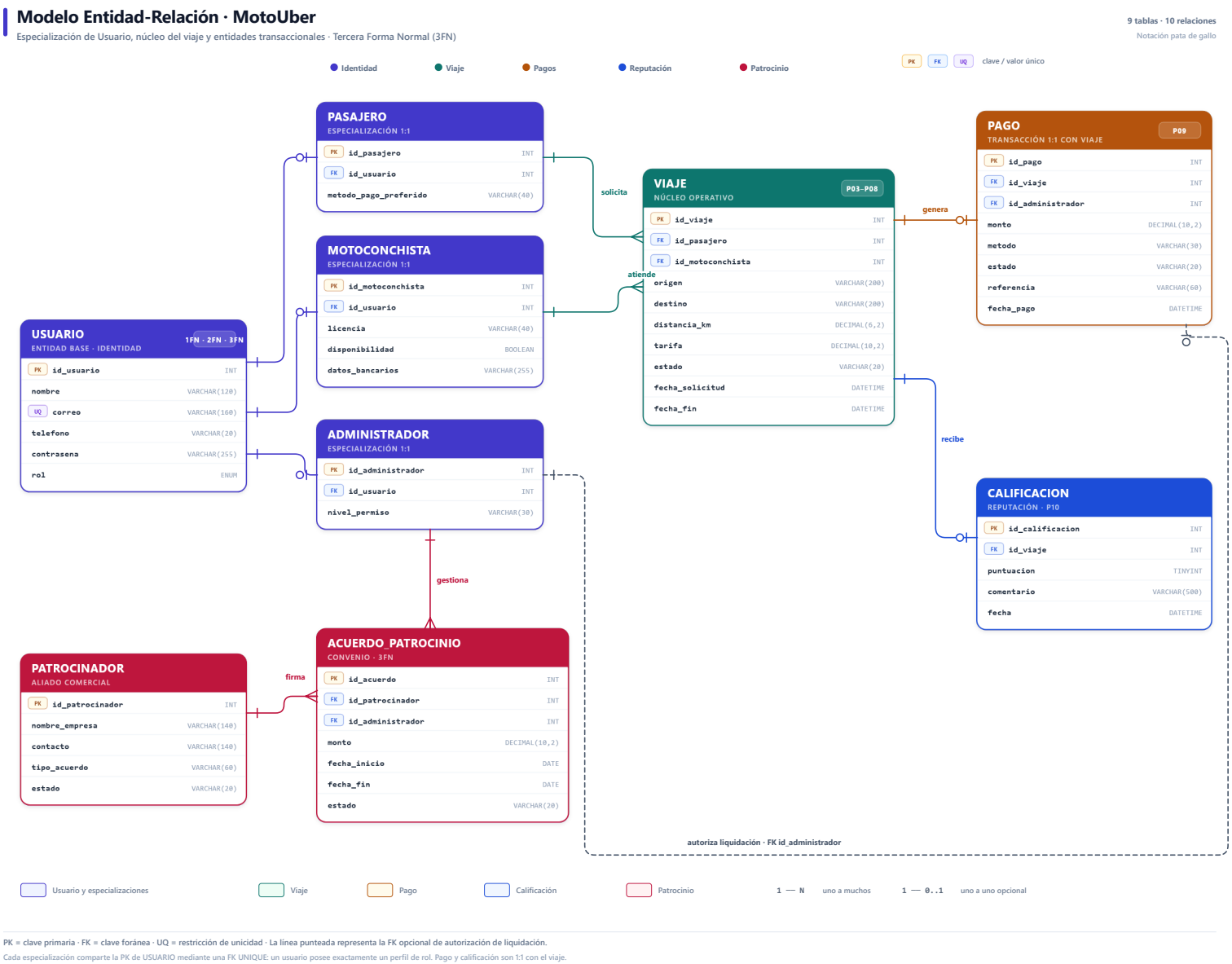


FIG. 1 **Modelo E-R de MotoUber.** Usuario es la entidad base; Pasajero, Motoconchista y Administrador son especializaciones 1:1. Viaje relaciona ambos extremos del servicio y concentra pago y calificación. Patrocinador y Acuerdo_Patrocinio quedan fuera del ciclo operativo.

Lectura de la pata de gallo: la barra perpendicular significa «uno»; el círculo, «opcional»; las tres líneas abiertas, «muchos». Así, `USUARIO 1 — 0..1 PASAJERO` se lee: un usuario puede tener cero o un perfil de pasajero, y cada pasajero pertenece a exactamente un usuario.

02 Diccionario de datos

Tipos, claves, nulabilidad y reglas de cada tabla. Los tipos son compatibles con MySQL 8.0 y PostgreSQL con cambios menores.

usuario 3FN

Entidad base · credenciales y rol

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_usuario	INT UNSIGNED	PK	No	Identificador único autoincremental.
nombre	VARCHAR(120)	—	No	Nombre completo o razón social de la persona.
correo	VARCHAR(160)	UQ	No	Credencial de acceso; restricción `UNIQUE` y validación de formato.
telefono	VARCHAR(20)	—	No	Contacto; se normaliza a formato internacional.
contrasena	VARCHAR(255)	—	No	Hash bcrypt o Argon2. Prohibido almacenar texto plano.
rol	ENUM	—	No	`pasajero`, `motoconchista` o `administrador`. Discriminador del perfil.

pasajero 3FN

Especialización 1:1 · datos propios del pasajero

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_pasajero	INT UNSIGNED	PK	No	Identificador del perfil de pasajero.
id_usuario	INT UNSIGNED	FK UQ	No	Referencia a `usuario.id_usuario`. `UNIQUE` garantiza la relación 1:1.
metodo_pago_preferido	VARCHAR(40)	—	Sí	`efectivo`, `tarjeta`, `transferencia` u otro medio habilitado.

motoconchista 3FN

Especialización 1:1 · datos operativos y bancarios

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_motoconchista	INT UNSIGNED	PK	No	Identificador del perfil de conductor.
id_usuario	INT UNSIGNED	FK UQ	No	Referencia a `usuario.id_usuario`; `UNIQUE` para la relación 1:1.
licencia	VARCHAR(40)	UQ	No	Número de licencia de conducir; `UNIQUE` para evitar duplicados.
disponibilidad	BOOLEAN	—	No	Estado persistido para recibir solicitudes; por defecto `TRUE`.
datos_bancarios	VARCHAR(255)	—	Sí	Referencia cifrada de la cuenta de liquidación. No se expone en la API pública.

administrador 3FN

Especialización 1:1 · permisos administrativos

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_administrador	INT UNSIGNED	PK	No	Identificador del perfil administrativo.
id_usuario	INT UNSIGNED	FK UQ	No	Referencia a `usuario.id_usuario`; `UNIQUE` para la relación 1:1.
nivel_permiso	ENUM	—	No	`basico`, `supervisor` o `total`. Controla operaciones sensibles.

patrocinador 3FN

Aliado comercial · no es un rol de acceso

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_patrocinador	INT UNSIGNED	PK	No	Identificador de la empresa patrocinadora.
nombre_empresa	VARCHAR(140)	—	No	Nombre comercial del aliado.
contacto	VARCHAR(140)	—	No	Persona, correo o teléfono de contacto.
tipo_acuerdo	VARCHAR(60)	—	No	Tipo de alianza preferente: `publicidad`, `descuento`, `patrocinio` u otro.
estado	VARCHAR(20)	—	No	`activo`, `inactivo` o `suspendido`.

viaje 3FN

Núcleo operativo · P03–P08

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_viaje	INT UNSIGNED	PK	No	Identificador del servicio solicitado.
id_pasajero	INT UNSIGNED	FK	No	Pasajero que solicita el viaje.
id_motoconchista	INT UNSIGNED	FK	Sí	Nulo mientras el viaje está en búsqueda; se completa al asignar.
origen	VARCHAR(200)	—	No	Dirección o punto de recogida.
destino	VARCHAR(200)	—	No	Dirección o punto de entrega.
distancia_km	DECIMAL(6,2)	—	Sí	Distancia estimada o confirmada; se completa al calcular la tarifa.
tarifa	DECIMAL(10,2)	—	Sí	Precio final en la moneda del sistema. Instantánea histórica, no calculada al leer.
estado	VARCHAR(20)	—	No	`solicitado`, `buscando`, `asignado`, `en_camino`, `en_curso`, `finalizado` o `cancelado`.
fecha_solicitud	DATETIME	—	No	Marca temporal de creación de la solicitud.
fecha_fin	DATETIME	—	Sí	Marca temporal de finalización o cancelación.

pago 3FN

Transacción 1:1 con viaje · P09

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_pago	INT UNSIGNED	PK	No	Identificador de la transacción.
id_viaje	INT UNSIGNED	FK UQ	No	UNIQUE: un viaje genera como máximo un registro de pago.
id_administrador	INT UNSIGNED	FK	Sí	Administrador que autoriza la liquidación o el reverso.
monto	DECIMAL(10,2)	—	No	Importe cobrado; restricción monto >= 0.
metodo	VARCHAR(30)	—	No	efectivo, tarjeta, transferencia o pasarela externa.
estado	VARCHAR(20)	—	No	pendiente, pagado, liquidado, revertido o cancelado.
referencia	VARCHAR(60)	UQ	Sí	Referencia de la pasarela; UNIQUE cuando existe para evitar doble cobro.
fecha_pago	DATETIME	—	Sí	Fecha efectiva del cobro o de la liquidación.

calificacion 3FN

Reputación 1:1 con viaje · P10

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_calificacion	INT UNSIGNED	PK	No	Identificador de la calificación.
id_viaje	INT UNSIGNED	FK UQ	No	UNIQUE: una sola calificación por viaje.
puntuacion	TINYINT	—	No	Entero entre 1 y 5; restricción CHECK.
comentario	VARCHAR(500)	—	Sí	Texto libre moderado por el administrador.
fecha	DATETIME	—	No	Fecha de registro de la calificación.

acuerdo_patrocinio 3FN

Convenio comercial · P11

CAMPO	TIPO	CLAVE	NULO	DESCRIPCIÓN / RESTRICCIÓN
id_acuerdo	INT UNSIGNED	PK	No	Identificador del convenio.
id_patrocinador	INT UNSIGNED	FK	No	Empresa que firma el acuerdo.
id_administrador	INT UNSIGNED	FK	No	Administrador responsable de la gestión.
monto	DECIMAL(10,2)	—	No	Valor económico del convenio; monto >= 0.
fecha_inicio	DATE	—	No	Inicio de vigencia.
fecha_fin	DATE	—	No	Fin de vigencia; debe ser posterior a fecha_inicio.
estado	VARCHAR(20)	—	No	borrador, activo, vencido o suspendido.

03 Relaciones y cardinalidades

La implementación física de cada relación se resuelve con claves foráneas; no hay relaciones muchos-a-muchos sin tabla intermedia.

RELACIÓN	ENTIDADES	CARDINALIDAD	IMPLEMENTACIÓN	REGLA DE INTEGRIDAD
Es un	Usuario → Pasajero	1 : 0..1	`pasajero.id_usuario` FK UNIQUE	`ON DELETE CASCADE`: al borrar el usuario se elimina su perfil.
Es un	Usuario → Motoconchista	1 : 0..1	`motoconchista.id_usuario` FK UNIQUE	`ON DELETE CASCADE`.
Es un	Usuario → Administrador	1 : 0..1	`administrador.id_usuario` FK UNIQUE	`ON DELETE CASCADE`.
Solicita	Pasajero → Viaje	1 : N	`viaje.id_pasajero` FK	`ON DELETE RESTRICT`: no se borra un pasajero con viajes.
Atiende	Motoconchista → Viaje	1 : N (opcional)	`viaje.id_motoconchista` FK NULL	`ON DELETE SET NULL`: el viaje conserva su historial si se elimina el conductor.
Genera	Viaje → Pago	1 : 0..1	`pago.id_viaje` FK UNIQUE	`ON DELETE RESTRICT`: un pago no se borra en cascada.
Recibe	Viaje → Calificación	1 : 0..1	`calificacion.id_viaje` FK UNIQUE	`ON DELETE CASCADE`: la calificación no existe sin el viaje.
Autoriza	Administrador → Pago	1 : N (opcional)	`pago.id_administrador` FK NULL	`ON DELETE SET NULL`: se conserva el pago y se pierde el responsable.
Firma	Patrocinador → Acuerdo	1 : N	`acuerdo.id_patrocinador` FK	`ON DELETE RESTRICT`: no se borra un patrocinador con convenios.
Gestiona	Administrador → Acuerdo	1 : N	`acuerdo.id_administrador` FK	`ON DELETE RESTRICT`: el convenio conserva un responsable.

⚠ **Decisión deliberada:** el administrador no se conecta con `viaje` mediante una FK. Su supervisión es una capacidad de rol (consulta y auditoría), no una relación de datos. Si en el futuro se requiere un responsable por viaje, se añadiría `id_administrador_supervisor` con su propia regla.

04 Normalización en Tercera Forma Normal

El objetivo no es «quitar tablas», sino asegurar que cada hecho del negocio se almacene una sola vez y que una actualización no pueda contradecir otra fila.

1FN · Atomicidad

Cada celda contiene un valor indivisible, no listas ni grupos repetidos. Cada fila se identifica por una clave primaria.

2FN · Dependencia total

Ningún atributo no clave depende solo de una parte de una clave compuesta. Si la clave es simple, la condición se cumple por construcción.

3FN · Sin transitivas

Ningún atributo no clave determina a otro atributo no clave. Los datos de otra entidad se alcanzan por clave foránea, no por copia.

UNF

Relación universal

Un solo registro de viaje con columnas repetidas para pagos, calificaciones y patrocinadores.

- Grupos repetidos
- Datos del pasajero y del conductor duplicados
- Datos del patrocinador mezclados con el viaje

1FN

Filas atómicas

Se separan los grupos repetidos en relaciones independientes y se asigna una clave a cada fila.

- `viaje_pago`
- `viaje_calificacion`
- `viaje_patrocinio`

2FN

Sin dependencias parciales

Con claves compuestas, los datos del pasajero dependen solo de `id_pasajero`, no de la clave del pago.

- Se separa `pasajero`
- Se separa `motoconchista`
- El viaje conserva solo sus propios hechos

3FN

Sin dependencias transitivas

El banco del conductor depende del conductor, no del viaje; el nombre del patrocinador depende del patrocinador, no del acuerdo.

- Se separa `pago`
- Se separa `calificacion`
- Se separan `patrocinador` y `acuerdo_patrocinio`

Caso de estudio: de una tabla universal a nueve relaciones

ETAPA	SITUACIÓN DE PARTIDA	ACCIÓN APLICADA	RESULTADO
UNF	`registro_viaje` con `pago_1_monto`, `pago_2_monto`, `calificacion_1`, `patrocinador_1`, además de nombre y correo del pasajero y nombre y licencia del conductor en cada fila.	Identificar grupos repetidos y atributos multivaluados.	Se reconoce que un viaje puede tener pagos, calificaciones y convenios asociados; no son columnas del viaje.
1FN	Columnas repetidas y listas dentro de una celda.	Separar cada grupo en una relación con su propia clave; una fila por pago, calificación o convenio.	`viaje_pago`, `viaje_calificacion` y `viaje_patrocinio` con valores atómicos.
2FN	Claves compuestas como `(id_viaje, id_pago)`; el nombre y el correo del pasajero dependen solo de `id_pasajero`.	Separar los datos que dependen de una sola parte de la clave: pasajero, conductor y viaje.	`pasajero`, `motoconchista` y `viaje` con claves simples y dependencias completas.
3FN	El banco del conductor viaja con el viaje; el nombre del patrocinador viaja con el acuerdo; el estado del pago viaja con el viaje.	Separar los hechos que dependen de otra entidad y dejar solo la clave foránea.	`pago`, `calificacion`, `patrocinador` y `acuerdo_patrocinio`; el viaje conserva solo sus atributos propios.

Antes y después: `viaje(id_viaje, pasajero_nombre, pasajero_correo, conductor_nombre, conductor_licencia, pago_monto, …)` se convierte en `viaje(id_viaje, id_pasajero, id_motoconchista, …)` más `pasajero`, `motoconchista` y `pago`. El nombre del pasajero se escribe una vez y se lee por `JOIN`.

Cumplimiento por tabla

Notación: `{X} → {Y}` significa que `X` determina funcionalmente a `Y`. Las claves candidatas se indican entre paréntesis.

TABLA	CLAVE PRIMARIA	DEPENDENCIAS FUNCIONALES	1FN	2FN	3FN	JUSTIFICACIÓN
usuario	id_usuario	`{id_usuario} → {nombre, correo, telefono, contrasena, rol}` `{correo} → {id_usuario}` (clave candidata)	Sí	Sí	Sí	Todos los atributos describen al usuario. El correo es una clave alterna, no un dato derivado.
pasajero	id_pasajero	`{id_pasajero} → {id_usuario, metodo_pago_preferido}` `{id_usuario} → {id_pasajero}` (1:1)	Sí	Sí	Sí	El perfil no repite nombre ni correo: los hereda por la FK a `usuario`.
motoconchista	id_motoconchista	`{id_motoconchista} → {id_usuario, licencia, disponibilidad, datos_bancarios}` `{id_usuario} → {id_motoconchista}` `{licencia} → {id_motoconchista}`	Sí	Sí	Sí	La licencia es clave candidata. Los datos bancarios dependen del conductor, no del viaje.
administrador	id_administrador	`{id_administrador} → {id_usuario, nivel_permiso}` `{id_usuario} → {id_administrador}`	Sí	Sí	Sí	El nivel de permiso pertenece al administrador; no se duplica en `usuario`.
patrocinador	id_patrocinador	`{id_patrocinador} → {nombre_empresa, contacto, tipo_acuerdo, estado}`	Sí	Sí	Sí	El tipo de alianza describe al patrocinador y no se repite en cada acuerdo.
viaje	id_viaje	`{id_viaje} → {id_pasajero, id_motoconchista, origen, destino, distancia_km, tarifa, estado, fecha_solicitud, fecha_fin}`	Sí	Sí	Sí	La tarifa es una instantánea del precio cobrado; no se recalcula desde la distancia al leer.
pago	id_pago	`{id_pago} → {id_viaje, id_administrador, monto, metodo, estado, referencia, fecha_pago}` `{id_viaje} → {id_pago}` (1:1)	Sí	Sí	Sí	Monto y estado pertenecen al acto de pago. La autorización del administrador es un hecho del pago.
calificacion	id_calificacion	`{id_calificacion} → {id_viaje, puntuacion, comentario, fecha}` `{id_viaje} → {id_calificacion}` (1:1)	Sí	Sí	Sí	La puntuación depende de la calificación; el viaje solo la identifica y la hace única.
acuerdo_patrocinio	id_acuerdo	`{id_acuerdo} → {id_patrocinador, id_administrador, monto, fecha_inicio, fecha_fin, estado}`	Sí	Sí	Sí	Monto y vigencia son del convenio. El tipo de alianza queda en el patrocinador para no duplicarlo.

Qué se evita con la 3FN

- **Duplicar el nombre del pasajero** en cada viaje: se cambia en un solo lugar.
- **Duplicar la licencia del conductor** en el historial de viajes.
- **Repetir el estado del pago** dentro de `viaje`: el pago tiene su propio ciclo.
- **Guardar el nombre del patrocinador** en cada acuerdo: se obtiene por `JOIN`.
- **Anomalías de borrado:** al eliminar un acuerdo no se pierde el patrocinador ni su historial.

Qué se conserva a propósito

- `viaje.tarifa` y `pago.monto`: no son el mismo hecho. La tarifa es el precio acordado; el monto es lo efectivamente cobrado, liquidado o revertido.
- `viaje.distancia_km`: es una medición del trayecto, no un atributo derivado de la tarifa.
- `pago.id_administrador`: registra quién autorizó; no es una relación administrativa con el viaje.
- `patrocinador.tipo_acuerdo`: describe la alianza preferente. Si el negocio exige un tipo por convenio, se mueve a `acuerdo_patrocinio`; nunca se duplica.

✓ **Verificación 3FN:** en las nueve tablas, la clave primaria es simple y todas las dependencias funcionales parten de ella o de una clave candidata declarada. No hay atributos no clave que determinen a otros atributos no clave.

05 Reglas de integridad y restricciones

Las reglas se implementan en la base de datos cuando son declarativas y en la capa de servicio cuando requieren contexto de negocio.

ID	REGLA	IMPLEMENTACIÓN	EFECTO ESPERADO
RI-01	Identidad única de cada fila.	`PRIMARY KEY` autoincremental en las nueve tablas.	No hay filas duplicadas ni identificadores reutilizados.
RI-02	Integridad referencial.	`FOREIGN KEY` con las acciones `CASCADE`, `SET NULL` o `RESTRICT` indicadas en cada relación.	No existen referencias huérfanas.
RI-03	Un correo, una cuenta.	`UNIQUE (correo)` en `usuario`.	El registro rechaza correos repetidos.
RI-04	Una licencia, un conductor.	`UNIQUE (licencia)` en `motoconchista`.	No se duplica el perfil de un conductor.
RI-05	Un usuario, un perfil de rol.	`UNIQUE (id_usuario)` en las tres especializaciones.	La relación 1:1 queda garantizada por el motor.
RI-06	Un viaje, un pago.	`UNIQUE (id_viaje)` en `pago`.	No se cobra dos veces el mismo viaje.
RI-07	Un viaje, una calificación.	`UNIQUE (id_viaje)` en `calificacion`.	No se duplica la reputación de un servicio.
RI-08	Estados válidos.	`CHECK` con la lista de estados de viaje, pago, patrocinador y acuerdo.	El motor rechaza estados fuera del catálogo.
RI-09	Puntuación entre 1 y 5.	`CHECK (puntuacion BETWEEN 1 AND 5)`.	No hay calificaciones fuera de escala.
RI-10	Montos y distancias no negativos.	`CHECK (>= 0)` en `monto`, `tarifa` y `distancia_km`.	Se evitan importes y distancias imposibles.
RI-11	Fechas coherentes.	`CHECK (fecha_fin >= fecha_inicio)` y `CHECK (fecha_fin >= fecha_solicitud)`.	Un convenio o viaje no termina antes de empezar.
RI-12	El rol de `usuario` coincide con el perfil creado.	Transacción de registro en P01 + vista de verificación `v_usuario_rol`.	No hay usuarios con rol «pasajero» y perfil de conductor, ni perfiles sin usuario.
RI-13	Transiciones de estado permitidas.	Máquina de estados en P05–P09; el API rechaza transiciones inválidas.	Un viaje cancelado no se marca como finalizado; un pago revertido no vuelve a «pendiente».
RI-14	Contraseña siempre como hash.	`VARCHAR(255)` + `CHECK (CHAR_LENGTH(contrasena) >= 60)` + bcrypt/Argon2 en la capa de servicio.	Se almacena un hash; nunca texto plano.
RI-15	La autorización de pago es opcional y auditables.	`pago.id_administrador` nullable con `ON DELETE SET NULL`.	El pago conserva su historial aunque se elimine el administrador.

Máquina de estados

ENTIDAD	DESDE	EVENTO	HACIA	PROCE
viaje	`solicitado`	registrar y buscar	`buscando`	P03–P(
viaje	`buscando`	asignar conductor	`asignado`	P05
viaje	`asignado`	conductor en camino	`en_camino`	P07
viaje	`en_camino`	iniciar trayecto	`en_curso`	P07
viaje	`en_curso`	finalizar	`finalizado`	P07–P(
viaje	`solicitado` … `en_camino`	cancelar	`cancelado`	P06
pago	`pendiente`	confirmar cobro	`pagado`	P09
pago	`pagado`	liquidar al conductor	`liquidado`	P09
pago	`pendiente` / `pagado`	cancelar	`cancelado`	P09
pago	`pagado` / `liquidado`	revertir por incidencia	`revertido`	P09

Por qué la consistencia de rol no es un `CHECK`

Un `CHECK` de MySQL solo puede evaluar columnas de la misma fila. La correspondencia entre `usuario.rol` y la existencia de la fila en `pasajero`, `motoconchista` o `administrador` cruza tres tablas.

Por eso P01 crea el usuario y su perfil en una sola transacción. La vista `v_usuario_rol` permite auditar desviaciones sin bloquear la operación:

```
SELECT id_usuario, correo, rol
FROM v_usuario_rol
WHERE (rol = 'pasajero' AND NOT es_pasajero)
OR (rol = 'motoconchista' AND NOT
es_motoconchista)
OR (rol = 'administrador' AND NOT
es_administrador);
```

Si la consulta devuelve filas, hay una inconsistencia que debe corregirse antes de continuar.

06 Script SQL listo para ejecutar

DDL para MySQL 8.0.16 o superior. Incluye tablas, claves, restricciones `CHECK`, índices de consulta y dos vistas de verificación.

⚠ **Antes de ejecutar:** el script elimina las tablas de `motouber` en orden inverso. No lo ejecutes sobre una base con datos reales sin una copia de seguridad. En MariaDB, verifica la versión y la sintaxis de `CHECK`.

motouber.sql · MySQL 8.0+

```
-- ========================================================================
-- MotoUber · Esquema relacional 3FN
-- Motor: MySQL 8.0.16+ (CHECK constraints)
-- Codificación: utf8mb4 / InnoDB
-- ========================================================================

CREATE DATABASE IF NOT EXISTS motouber
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE motouber;

-- Orden inverso de dependencias
DROP TABLE IF EXISTS acuerdo_patrocinio;
DROP TABLE IF EXISTS calificacion;
DROP TABLE IF EXISTS pago;
DROP TABLE IF EXISTS viaje;
DROP TABLE IF EXISTS patrocinador;
DROP TABLE IF EXISTS administrador;
DROP TABLE IF EXISTS motoconchista;
DROP TABLE IF EXISTS pasajero;
DROP TABLE IF EXISTS usuario;

-- ------------------------------------------------------------------------
-- 1. USUARIO · identidad base y credenciales
-- ------------------------------------------------------------------------
CREATE TABLE usuario (
  id_usuario   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nombre       VARCHAR(120) NOT NULL,
  correo       VARCHAR(160) NOT NULL,
  telefono     VARCHAR(20)  NOT NULL,
  contrasena   VARCHAR(255) NOT NULL,
  rol          ENUM('pasajero','motoconchista','administrador') NOT NULL,
  PRIMARY KEY (id_usuario),
  UNIQUE KEY uq_usuario_correo (correo),
  CONSTRAINT chk_usuario_correo CHECK (correo LIKE '%_@_%._%'),
  CONSTRAINT chk_usuario_hash CHECK (CHAR_LENGTH(contrasena) >= 60)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ------------------------------------------------------------------------
-- 2. PASAJERO · especialización 1:1
-- ------------------------------------------------------------------------
CREATE TABLE pasajero (
  id_pasajero            INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario             INT UNSIGNED NOT NULL,
  metodo_pago_preferido  VARCHAR(40) NULL,
  PRIMARY KEY (id_pasajero),
  UNIQUE KEY uq_pasajero_usuario (id_usuario),
  CONSTRAINT fk_pasajero_usuario FOREIGN KEY (id_usuario)
    REFERENCES usuario (id_usuario)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ------------------------------------------------------------------------
-- 3. MOTOCONCHISTA · especialización 1:1
-- ------------------------------------------------------------------------
CREATE TABLE motoconchista (
  id_motoconchista  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario        INT UNSIGNED NOT NULL,
```

```
  licencia          VARCHAR(40) NOT NULL,
  disponibilidad    BOOLEAN NOT NULL DEFAULT TRUE,
  datos_bancarios   VARCHAR(255) NULL,
  PRIMARY KEY (id_motoconchista),
  UNIQUE KEY uq_motoconchista_usuario (id_usuario),
  UNIQUE KEY uq_motoconchista_licencia (licencia),
  CONSTRAINT fk_motoconchista_usuario FOREIGN KEY (id_usuario)
    REFERENCES usuario (id_usuario)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- 4. ADMINISTRADOR · especialización 1:1
-- ---------------------------------------------------------------------
CREATE TABLE administrador (
  id_administrador  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario        INT UNSIGNED NOT NULL,
  nivel_permiso     ENUM('basico','supervisor','total') NOT NULL DEFAULT 'basico',
  PRIMARY KEY (id_administrador),
  UNIQUE KEY uq_administrador_usuario (id_usuario),
  CONSTRAINT fk_administrador_usuario FOREIGN KEY (id_usuario)
    REFERENCES usuario (id_usuario)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- 5. PATROCINADOR · aliado comercial
-- ---------------------------------------------------------------------
CREATE TABLE patrocinador (
  id_patrocinador  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nombre_empresa   VARCHAR(140) NOT NULL,
  contacto         VARCHAR(140) NOT NULL,
  tipo_acuerdo     VARCHAR(60)  NOT NULL,
  estado           VARCHAR(20)  NOT NULL DEFAULT 'activo',
  PRIMARY KEY (id_patrocinador),
  CONSTRAINT chk_patrocinador_estado
    CHECK (estado IN ('activo','inactivo','suspendido'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- 6. VIAJE · núcleo operativo
-- ---------------------------------------------------------------------
CREATE TABLE viaje (
  id_viaje          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_pasajero       INT UNSIGNED NOT NULL,
  id_motoconchista  INT UNSIGNED NULL,
  origen            VARCHAR(200) NOT NULL,
  destino           VARCHAR(200) NOT NULL,
  distancia_km      DECIMAL(6,2)  NULL,
  tarifa            DECIMAL(10,2) NULL,
  estado            VARCHAR(20)  NOT NULL DEFAULT 'solicitado',
  fecha_solicitud   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_fin         DATETIME NULL,
  PRIMARY KEY (id_viaje),
  KEY idx_viaje_estado (estado, fecha_solicitud),
  CONSTRAINT fk_viaje_pasajero FOREIGN KEY (id_pasajero)
    REFERENCES pasajero (id_pasajero)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_viaje_motoconchista FOREIGN KEY (id_motoconchista)
    REFERENCES motoconchista (id_motoconchista)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT chk_viaje_estado CHECK (estado IN
```

```
    ('solicitado','buscando','asignado','en_camino','en_curso','finalizado','cancelado')),
  CONSTRAINT chk_viaje_distancia CHECK (distancia_km IS NULL OR distancia_km >= 0),
  CONSTRAINT chk_viaje_tarifa CHECK (tarifa IS NULL OR tarifa >= 0),
  CONSTRAINT chk_viaje_fechas CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_solicitud)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- 7. PAGO · transacción 1:1 con viaje
-- ---------------------------------------------------------------------
CREATE TABLE pago (
  id_pago           INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_viaje          INT UNSIGNED NOT NULL,
  id_administrador  INT UNSIGNED NULL,
  monto             DECIMAL(10,2) NOT NULL,
  metodo            VARCHAR(30) NOT NULL,
  estado            VARCHAR(20) NOT NULL DEFAULT 'pendiente',
  referencia        VARCHAR(60) NULL,
  fecha_pago        DATETIME NULL,
  PRIMARY KEY (id_pago),
  UNIQUE KEY uq_pago_viaje (id_viaje),
  UNIQUE KEY uq_pago_referencia (referencia),
  KEY idx_pago_estado (estado),
  CONSTRAINT fk_pago_viaje FOREIGN KEY (id_viaje)
    REFERENCES viaje (id_viaje)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_pago_administrador FOREIGN KEY (id_administrador)
    REFERENCES administrador (id_administrador)
    ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT chk_pago_monto CHECK (monto >= 0),
  CONSTRAINT chk_pago_estado CHECK (estado IN
    ('pendiente','pagado','liquidado','revertido','cancelado'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- 8. CALIFICACION · reputación 1:1 con viaje
-- ---------------------------------------------------------------------
CREATE TABLE calificacion (
  id_calificacion  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_viaje         INT UNSIGNED NOT NULL,
  puntuacion       TINYINT UNSIGNED NOT NULL,
  comentario       VARCHAR(500) NULL,
  fecha            DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_calificacion),
  UNIQUE KEY uq_calificacion_viaje (id_viaje),
  CONSTRAINT fk_calificacion_viaje FOREIGN KEY (id_viaje)
    REFERENCES viaje (id_viaje)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT chk_calificacion_puntuacion
    CHECK (puntuacion BETWEEN 1 AND 5)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- 9. ACUERDO_PATROCINIO · convenio comercial
-- ---------------------------------------------------------------------
CREATE TABLE acuerdo_patrocinio (
  id_acuerdo        INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_patrocinador   INT UNSIGNED NOT NULL,
  id_administrador  INT UNSIGNED NOT NULL,
  monto             DECIMAL(10,2) NOT NULL,
  fecha_inicio      DATE NOT NULL,
  fecha_fin         DATE NOT NULL,
  estado            VARCHAR(20) NOT NULL DEFAULT 'borrador',
```

```
  PRIMARY KEY (id_acuerdo),
  KEY idx_acuerdo_vigencia (fecha_inicio, fecha_fin),
  CONSTRAINT fk_acuerdo_patrocinador FOREIGN KEY (id_patrocinador)
    REFERENCES patrocinador (id_patrocinador)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_acuerdo_administrador FOREIGN KEY (id_administrador)
    REFERENCES administrador (id_administrador)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_acuerdo_monto CHECK (monto >= 0),
  CONSTRAINT chk_acuerdo_fechas CHECK (fecha_fin >= fecha_inicio),
  CONSTRAINT chk_acuerdo_estado CHECK (estado IN
    ('borrador','activo','vencido','suspendido'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- ---------------------------------------------------------------------
-- Vistas de verificación y reporte
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_usuario_rol AS
SELECT
  u.id_usuario,
  u.correo,
  u.rol,
  (p.id_pasajero IS NOT NULL)       AS es_pasajero,
  (m.id_motoconchista IS NOT NULL)  AS es_motoconchista,
  (a.id_administrador IS NOT NULL)  AS es_administrador
FROM usuario u
LEFT JOIN pasajero p       ON p.id_usuario = u.id_usuario
LEFT JOIN motoconchista m  ON m.id_usuario = u.id_usuario
LEFT JOIN administrador a  ON a.id_usuario = u.id_usuario;

CREATE OR REPLACE VIEW v_acuerdo_vigente AS
SELECT
  ac.id_acuerdo,
  pa.nombre_empresa,
  ac.monto,
  ac.fecha_inicio,
  ac.fecha_fin,
  CASE
    WHEN CURDATE() BETWEEN ac.fecha_inicio AND ac.fecha_fin THEN 'vigente'
    ELSE 'fuera_de_vigencia'
  END AS vigencia
FROM acuerdo_patrocinio ac
JOIN patrocinador pa ON pa.id_patrocinador = ac.id_patrocinador
WHERE ac.estado = 'activo';
```

Cómo ejecutarlo

1. Abre MySQL Workbench o la consola `mysql`.
2. Conéctate con un usuario que pueda crear bases y tablas.
3. Pega el script completo y ejecútalo de una vez.
4. Verifica con `SHOW TABLES;` que aparezcan las nueve tablas y las dos vistas.

Consultas de verificación

```
-- Usuarios con rol y perfil inconsistentes
SELECT * FROM v_usuario_rol
WHERE (rol = 'pasajero' AND NOT es_pasajero)
OR (rol = 'motoconchista' AND NOT
es_motoconchista)
OR (rol = 'administrador' AND NOT
es_administrador);
-- Viajes finalizados sin pago
SELECT v.id_viaje FROM viaje v
LEFT JOIN pago p ON p.id_viaje = v.id_viaje
WHERE v.estado = 'finalizado' AND p.id_pago IS
NULL;
```

07

Supuestos, decisiones y referencias

Límites explícitos del modelo y criterios que permiten verificarlo sin depender de una implementación concreta.

Supuestos del negocio

- Un viaje genera como máximo un pago y una calificación; si el negocio necesita pagos parciales, se cambia la cardinalidad y se añade una tabla de cuotas.
- El patrocinador no inicia sesión: es un aliado comercial, no un rol de acceso.
- La supervisión del administrador es una capacidad de rol; no se asigna un administrador obligatorio a cada viaje.
- La moneda es única y parametrizable; los importes se guardan con dos decimales.
- La distancia se expresa en kilómetros y la tarifa es el precio final cobrado, no una fórmula recalculada al leer.
- Las fechas se almacenan en UTC y se presentan en la zona horaria del usuario.

Decisiones de diseño

- **Superusuario + subtipos 1:1:** evita duplicar credenciales y permite permisos por rol.
- **Claves sustitutas:** cada tabla usa `INT UNSIGNED AUTO_INCREMENT`; las claves naturales se protegen con `UNIQUE`.
- **Pago y calificación separados:** tienen ciclos de vida y responsables distintos.
- **Patrocinio independiente:** no contamina el ciclo operativo del viaje.
- **Sin borrado lógico:** las FK `RESTRICT` protegen el historial; si se requiere baja lógica, se añade `activo` o `deleted_at` con una migración.

✓ **Checklist de verificación:** nueve tablas, diez relaciones, una PK por tabla, una FK por relación, `UNIQUE` en las relaciones 1:1 y en las claves naturales, `CHECK` en estados, fechas, montos y puntuación, y acciones `ON DELETE` coherentes con la historia del negocio.

Referencias

- Chen, P. P. (1976). *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems.
- Codd, E. F. (1970, 1971, 1972). Artículos fundacionales sobre el modelo relacional y las formas normales.
- Elmasri, R. y Navathe, S. B. *Fundamentals of Database Systems*. Capítulos de modelo relacional, normalización y diseño lógico.

- Date, C. J. *An Introduction to Database Systems*. Dependencias funcionales y descomposición sin pérdida.
- Oracle. *MySQL 8.0 Reference Manual*: CHECK constraints y claves foráneas InnoDB.
- OWASP. *Password Storage Cheat Sheet*: bcrypt, Argon2 y buenas prácticas.
- W3C. *Scalable Vector Graphics (SVG) 2*: especificación.
- Yourdon, E. y DeMarco, T. *Structured Analysis and System Specification*. Notación de flujos de datos utilizada en el dossier.